

Design Document

Automated Dependency Monitoring CI System (sw-deps-monitor)

Design Document	1
Automated Dependency Monitoring CI System (sw-deps-monitor)	1
1. Introduction	2
1.1 Purpose.....	3
2. Software and Driver Required	3
3. Software Architecture	3
4. Software Module Functionality	3
4.1 Generate Changed Files List and Changed Libraries List.....	4
4.2 Check If Files are Associated with a Library	4
4.3 Unexpected Dependencies Check.....	4
4.4 Update Actual Deps List and Circular Dependencies Check	4
4.5 Commit Dependency Updates (If Needed)	4
4.6 Check if PR Branch is Up to Date with Base Branch	5
5. Software Behavior at Critical Occurrences	5
5.1 Dependency Violation Detected	6
5.2 Missing or Incorrect .deps File	6
5.3 Python Script Failure	6
5.4 YAML / Workflow Misconfiguration.....	6
5.5 GitHub API / PR Creation Failure.....	6
5.6 Network / Connectivity Issues (Runner)	7
5.7 Central Python Scripts Not Found	7
5.8 Dependency Changes Detected (Auto Update Scenario)	7

1. Introduction

1.1 Purpose

The **primary purpose** of the *SW Deps Monitor* project is to provide an automated system for **monitoring and enforcing software library dependencies** within a codebase.

It aims to:

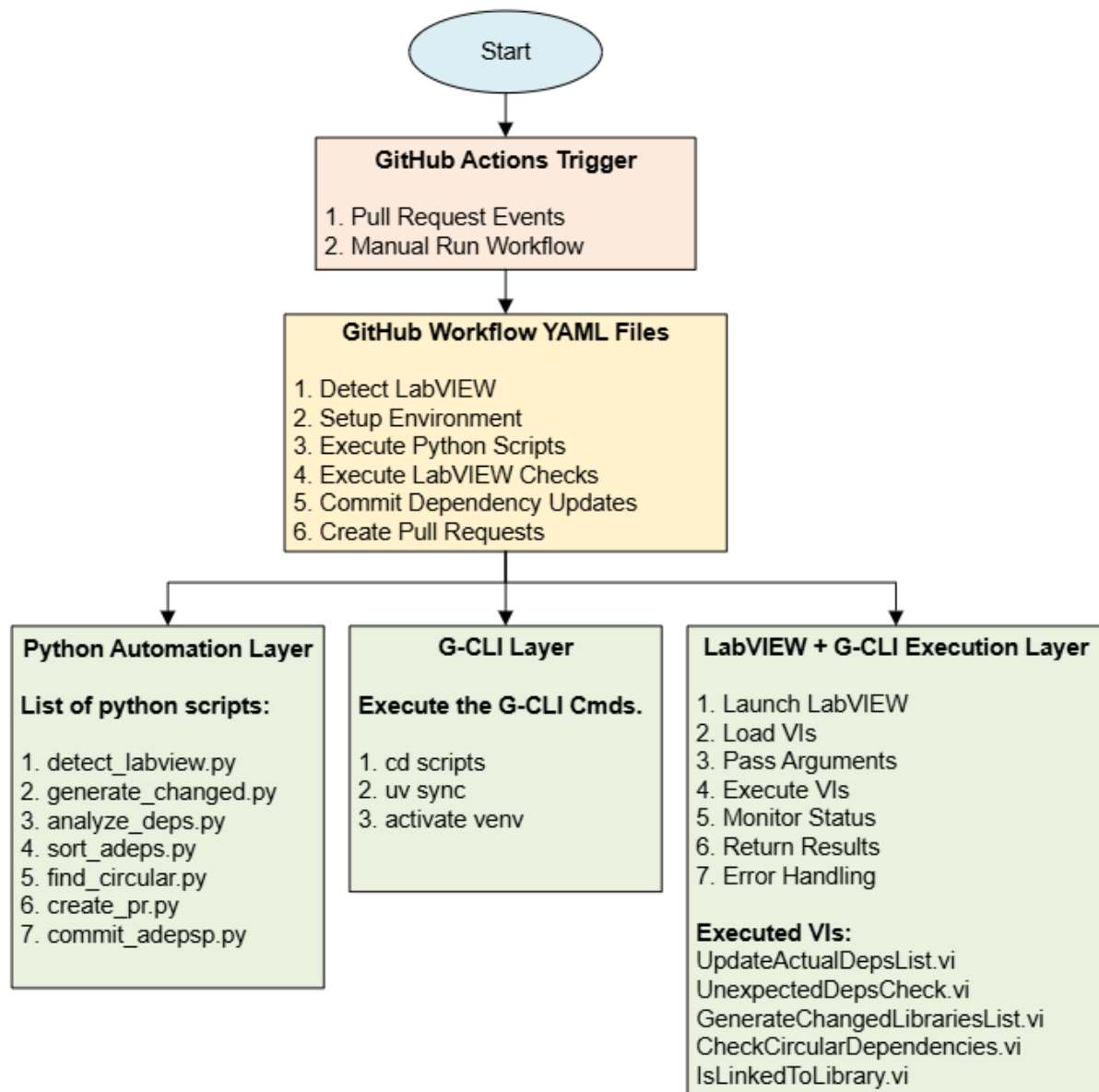
- Ensure that software systems maintain a **clean and structured architecture**
- Prevent the introduction of **undesired or unintended dependencies** between modules
- Enable teams to **enforce architectural rules consistently**
- Integrate dependency validation directly into the **CI/CD pipeline** for continuous verification

2. Software and Driver Required

- LabVIEW 2019 or Later (Supports both 32 & 64 bit)
- DepsWatch VI PAcKage from [repo](#)
- Python 3.12 or later
- Git

3. Software Architecture

SW Deps Monitor provides an automated workflow to enforce software dependency rules across the development lifecycle. It uses a combination of LabVIEW (CICD VIP), GitHub Actions, and Python scripts to validate dependencies against predefined architectural rules (.deps). During development, any violation is automatically detected in CI/CD, blocking incorrect changes and ensuring that the system remains modular, consistent, and aligned with the intended architecture.



4. Software Module Functionality

4.1 Generate Changed Files List and Changed Libraries List

This module identifies all the files modified in the current commit or pull request and maps those changes to the corresponding libraries or modules. It produces:

A list of **changed files**

A list of **impacted libraries**

This helps limit dependency validation only to affected areas and improves analysis efficiency.

4.2 Check If Files are Associated with a Library

This module validates whether the modified files are properly associated with a defined library or module in the project structure. If any file is not mapped to a known library, it is flagged for review.

This ensures:

- Proper project organization
- Accurate dependency tracking

4.3 Unexpected Dependencies Check

This module compares actual dependencies in the codebase with the expected rules defined in .deps files. If any unauthorized or unexpected dependency is detected, it is flagged.

If violations exist:

- CI/CD pipeline fails
- Pull request is blocked
- This enforces strict architectural compliance.

4.4 Update Actual Deps List and Circular Dependencies Check

This module updates the ActualDepsList.adeps file to reflect the current dependency structure. It also performs circular dependency analysis to detect loops between modules.

If circular dependencies are found:

- They are reported as issues
- Developers are expected to refactor
- This ensures maintainable and loosely coupled architecture.

4.5 Commit Dependency Updates (If Needed)

If there are valid changes in dependencies (without violations), this module commits the updated ActualDepsList.adepts file to a new branch.

This ensures:

- Dependency records stay up to date
- Changes are tracked in version control

4.6 Check if PR Branch is Up to Date with Base Branch

Before creating or updating a pull request, this module verifies whether the PR branch is synchronized with the latest base branch (e.g., main).

If the branch is outdated:

- It requires rebasing or merging
- Prevents conflicts and stale updates

5. Software Behavior at Critical Occurrences

5.1 Dependency Violation Detected

When a dependency violation occurs, the Python validation scripts compare the actual dependencies with the expected rules defined in the `.deps` file and detect any mismatch. Once identified, the CI/CD pipeline immediately marks the build as **FAILED**, and the corresponding pull request is **blocked from merging**. This ensures that no code violating the defined architecture is integrated into the codebase, thereby preventing architectural deviations and maintaining strict compliance with dependency rules.

5.2 Missing or Incorrect `.deps` File

If the `.deps` file is missing or incorrectly configured, the validation step identifies this issue during execution. The pipeline then shows error message and fails the build to enforce proper dependency rule definition. This behavior ensures that development teams always maintain accurate architectural definitions and prevents undefined or uncontrolled dependency behavior in the system.

5.3 Python Script Failure

In scenarios where the Python scripts responsible for dependency analysis fail to execute (due to errors, missing modules, or environment issues), the CI/CD workflow immediately stops execution. Detailed error logs are generated to assist in debugging. This prevents incomplete or incorrect validation from taking place and ensures that the system remains reliable by not allowing uncertain results.

5.4 YAML / Workflow Misconfiguration

If there is a misconfiguration in the GitHub Actions workflow YAML files, the pipeline may fail either at the trigger stage or during execution. In such cases, no dependency validation is performed, and the error is clearly visible in the workflow logs. This behavior ensures that workflow issues are corrected before the system resumes enforcement, preventing silent failures that could otherwise lead to undetected issues.

5.5 GitHub API / PR Creation Failure

When there are issues with GitHub API interactions (such as SSL problems, invalid tokens, or network issues), the automatic pull request creation process may fail. However, this does not affect the core dependency of validation. The system logs the error related to PR creation, but the validation result (pass or fail) is still enforced. This ensures that governance remains intact while only the automation aspect is temporarily impacted.

5.6 Network / Connectivity Issues (Runner)

If the CI/CD runner experiences network or connectivity issues, the job execution may fail during the pipeline run. In such cases, the job must be retried manually or automatically depending on the setup. The system ensures that no incorrect validation results are accepted during such failures, prioritizing architectural integrity over execution availability.

5.7 Central Python Scripts Not Found

If the Python scripts are not found in the configured path (as defined in PathConfigs.ini), the system terminates execution with an error. This prevents the pipeline from proceeding with incomplete or incorrect dependency analysis. It enforces proper environment setup and ensures that all required components are available before validation begins.

5.8 Dependency Changes Detected (Auto Update Scenario)

When the system detects valid but updated dependency structures (i.e., no violations but differences from the current .adepts file), it automatically generates a pull request to update the ActualDepsList.adepts file. This ensures that dependency records remain accurate and up to date while maintaining full transparency and enabling proper review of changes before they are merged.